Reviewer Pack — Minimal Order of Noncrossing Partitions and Communication Co…

Entry b793652ac874 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 05:12:06 UTC

Minimal Order of Noncrossing Partitions and Communication Complexity Rank Variance

Entry ID: `b793652ac874` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 05:12:06 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Enumeration (Noncrossing Partitions) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every instance of a communication problem, the minimal order of noncrossing partitions associated with its input corresponds to the variance in the ranks of the associated matrix representations of communication protocols.

Rationale (proposer's reasoning):

> Noncrossing partitions have been used to model certain combinatorial problems and may provide a way to analyze communication complexity. The minimal order could capture essential features of the input that influence the communication complexity, potentially providing new insights into lower bounds for communication complexity.

Taxonomy category: `COMBINATORICS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bdd5e42cee390233`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all communication problem instances, the correlation coefficient between the minimal order of noncrossing partitions and the variance in matrix ranks exceeds 0.7 with a p-value ≤ 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"noncrossing partitions" AND "communication complexity rank variance"` - `"matrix representation" AND "communication protocol" AND noncrossing` - `"minimal order" noncrossing partitions OR "matrix rank variance"`

Top relevant hits considered: - [http://arxiv.org/abs/1604.06009v2] Oriented Flip Graphs and Noncrossing Tree Partitions - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface - [http://arxiv.org/abs/1904.05573v3] k-Indivisible Noncrossing Partitions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def noncrossing_partitions(n):
        if n == 0:
            return [[]]
        partitions = []
        for i in range(1, n):
            for partition in noncrossing_partitions(i):
                new_partition = partition + [[j+i for j in partition[-1]]]
                partitions.append(new_partition)
        return partitions

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        if m != n:
            raise ValueError("Matrix must be square")

        # Gaussian elimination
        for i in range(n):
            # Find pivot
            max_row = i
            for r in range(i+1, n):
                if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                    max_row = r
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
```

```
            # Eliminate
            if matrix[i][i] == 0:
                continue
            for j in range(i+1, n):
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]

        # Count non-zero rows
        rank = sum(1 for row in matrix if any(row))
        return rank

    def communication_protocol_matrix(instance_size):
        # Placeholder for actual protocol matrix generation
        return [[random.randint(0, 1) for _ in range(instance_size)] for _ in range(instance_size)

    instance_sizes = [5, 10, 15, 20, 30, 40]
    results = []

    for n in instance_sizes:
        if n * (n - 1) // 2 < len(results):
            continue

        matrix = communication_protocol_matrix(n)
        rank = matrix_rank(matrix)
        noncrossing_partition_order = len(noncrossing_partitions(n))

        results.append({
            "instance_size": n,
            "noncrossing_partition_order": noncrossing_partition_order,
            "matrix_rank": rank
        })

    if not results:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    n_max = max(result["instance_size"] for result in results)
    correlation_values = [result["noncrossing_partition_order"] / result["matrix_rank"] for result
    mean_correlation = sum(correlation_values) / len(correlation_values)
    variance = sum((x - mean_correlation) ** 2 for x in correlation_values) / len(correlation_valu

    return {
        "metric_name": "correlation",
        "metric_value": mean_correlation,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": mean_correlation > 0.7,
        "counterexample": ""
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the correlation coefficient and p-value could not be calculated to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13358
2	propose	ollama_remote	glm4:latest	0	0	12258
3	preregistration	ollama_remote	glm4:latest	0	0	18587
4	novelty	ollama_remote	glm4:latest	0	0	10463
5	novelty	ollama_remote	glm4:latest	0	0	14231
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12365

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13155
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13566
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11883
10	judge	ollama_remote	glm4:latest	0	0	12959

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132826 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b793652ac874.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b793652ac874.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b793652ac874.tar.gz` (if generated)